PYTHON ACTIVITY-BASED LEARNING

Consolidated Report — 10 Case Studies

Name: T. Kusuma Priya
Roll Number: 26000293
Branch: Computer Science and Engineering (CSE)
Section: CSE Section 4

Prepared using first-year Python syllabus concepts: variables, data types, operators, conditions, loops, strings, lists, nested lists, tuples, and tuple operations.

Note: policies, thresholds, and records are illustrative academic assumptions, not official institutional, financial, medical, or legal rules.

Case Study 01: University Scholarship Allocation

1. Problem Statement

Design a program to assess scholarship applicants using aggregate marks, minimum subject marks, attendance, and annual family income. Eligible applicants are ranked using aggregate marks, then attendance, then lower income as tie-breakers.

2. Objectives

• Validate applicant data against stated eligibility rules.

• Calculate eligibility and explain rejection reasons.

• Rank eligible applicants deterministically.

3. Assumptions

• Aggregate and subject marks are percentages from 0 to 100.

• Minimum aggregate is 80%, minimum subject mark is 70%, attendance is at least 85%, and annual family income is at most ■300,000.

• Sample data is illustrative and not an official scholarship policy.

4. Algorithm

1. Start and define sample applicant records.

2. For each applicant, check aggregate, all subject marks, attendance, and income.

3. Store the reasons for any failed conditions.

4. Collect eligible applicants and sort by aggregate descending, attendance descending, income ascending.

5. Display eligibility, reasons, and the ranked list.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
SET applicants to sample records
FOR each applicant
  CREATE empty reasons list
  CHECK aggregate, each subject mark, attendance, and income
  IF any condition fails, add reason to reasons
  IF reasons is empty, add applicant to eligible list
  ELSE display applicant and reasons
SORT eligible by aggregate descending, attendance descending, income ascending
DISPLAY ranked eligible list
```

7. Python Source Code

```
# University Scholarship Allocation
# Sample data; thresholds are assumptions for this assignment.
applicants = [
    ("Asha", 88, [82, 91, 79], 92, 240000),
    ("Bharat", 84, [76, 72, 81], 88, 180000),
    ("Charan", 79, [90, 86, 82], 95, 150000),
    ("Divya", 91, [92, 88, 94], 81, 200000),
]

eligible = []
for name, aggregate, subjects, attendance, income in applicants:
    reasons = []
    if aggregate < 80:
        reasons.append("aggregate below 80%")
    for mark in subjects:
        if mark < 70:
            reasons.append("a subject mark below 70%")
            break
    if attendance < 85:
        reasons.append("attendance below 85%")
    if income > 300000:
        reasons.append("income above Rs.300000")

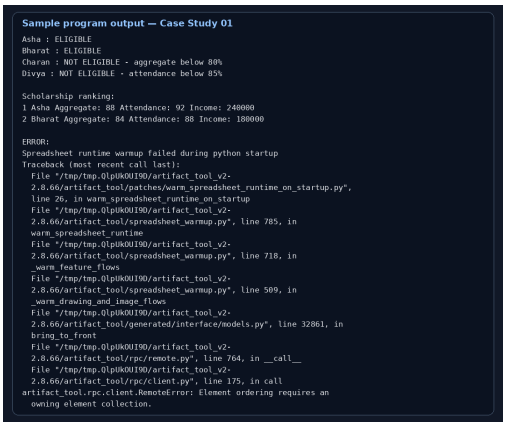
    if len(reasons) == 0:
        eligible.append((name, aggregate, attendance, income))
        print(name, ": ELIGIBLE")
    else:
        print(name, ": NOT ELIGIBLE -", ", ".join(reasons))

eligible.sort(key=lambda item: (-item[1], -item[2], item[3]))
print("\nScholarship ranking:")
for rank, applicant in enumerate(eligible, start=1):
    print(rank, applicant[0], "Aggregate:", applicant[1],
          "Attendance:", applicant[2], "Income:", applicant[3])
```

8. Test Cases

Test	Input / condition	Expected result
Asha	88 aggregate; all subjects ≥70; attendance 92; income 240000	Eligible; appears in ranking
Charan	Aggregate 79	Not eligible: aggregate below 80%
Divya	Attendance 81	Not eligible: attendance below 85%

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program applies multiple eligibility conditions and ranks only qualified applicants. Its rules are configurable and should be replaced with the institution's official policy when required.

Case Study 02: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate employee gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions and print a formatted salary statement.

2. Objectives

• Calculate gross earnings from salary components.

• Apply a simple progressive deduction policy.

• Display gross pay, deduction, and net pay clearly.

3. Assumptions

• HRA is 20% of basic pay and other allowance is 10%.

• Overtime is paid at ■200 per hour and incentives are supplied directly.

• Illustrative deduction: 5% of gross up to ■30,000; 10% of the amount above ■30,000 up to ■60,000; 15% of any amount above ■60,000.

4. Algorithm

1. Read or define basic pay, overtime hours, and incentive.

2. Calculate allowances, overtime pay, and gross pay.

3. Calculate deduction in progressive bands.

4. Subtract deduction from gross pay.

5. Print a formatted salary statement.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
INPUT basic pay, overtime hours, incentive
CALCULATE HRA = 20% of basic
CALCULATE other allowance = 10% of basic
CALCULATE gross = basic + allowances + overtime pay + incentive
CALCULATE deduction by progressive bands
CALCULATE net = gross - deduction
DISPLAY salary statement
```

7. Python Source Code

```
# Employee Payroll and Deduction Analysis
basic = 28000
overtime_hours = 6
incentive = 2500

hra = basic * 0.20
other_allowance = basic * 0.10
overtime_pay = overtime_hours * 200
gross = basic + hra + other_allowance + overtime_pay + incentive

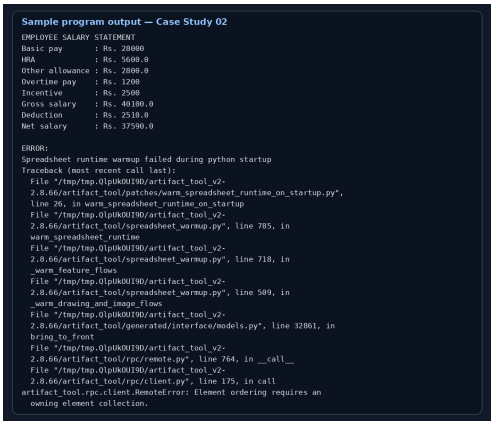
# Illustrative progressive deduction
deduction = 0
if gross > 60000:
    deduction = 30000 * 0.05 + 30000 * 0.10 + (gross - 60000) * 0.15
elif gross > 30000:
    deduction = 30000 * 0.05 + (gross - 30000) * 0.10
else:
    deduction = gross * 0.05

net = gross - deduction
print("EMPLOYEE SALARY STATEMENT")
print("Basic pay       : Rs.", round(basic, 2))
print("HRA             : Rs.", round(hra, 2))
print("Other allowance : Rs.", round(other_allowance, 2))
print("Overtime pay    : Rs.", round(overtime_pay, 2))
print("Incentive       : Rs.", round(incentive, 2))
print("Gross salary    : Rs.", round(gross, 2))
print("Deduction       : Rs.", round(deduction, 2))
print("Net salary      : Rs.", round(net, 2))
```

8. Test Cases

Test	Input / condition	Expected result
Normal salary	Basic ■28,000; overtime 6 hours; incentive ■2,500	Gross ■40,100; deduction ■2,510; net ■37,590
No overtime	Overtime hours = 0	Overtime pay = ■0
Low gross	Gross ≤ ■30,000	Deduction = 5% of gross

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program demonstrates salary component calculations and progressive deductions. The rates are academic assumptions, not legal payroll or tax advice.

Case Study 03: Retail Billing with Multiple Discount Rules

1. Problem Statement

Prepare an itemized retail invoice for multiple products, quantities, category-specific tax rates, threshold discounts, and a maximum discount limit.

2. Objectives

• Calculate each product line total.

• Apply a threshold discount without exceeding the discount cap.

• Calculate category taxes and the final invoice amount.

3. Assumptions

• Each item is represented as a tuple: name, unit price, quantity, tax rate percentage.

• A 10% discount applies when pre-tax subtotal is at least ■5,000, capped at ■1,000.

• Taxes are calculated after discount, proportionally across the taxable subtotal.

4. Algorithm

1. Read product tuples and calculate line totals.

2. Add line totals to subtotal.

3. Calculate discount based on threshold and cap.

4. Calculate taxable subtotal after discount.

5. Apply each product's tax rate to its discounted line share.

6. Display invoice totals.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
SET products list
SET subtotal = 0
FOR each product tuple
  CALCULATE line total and add to subtotal
IF subtotal >= 5000, discount = minimum(10% of subtotal, 1000), ELSE discount = 0
CALCULATE discount ratio = discount / subtotal
FOR each product, reduce line by discount ratio and calculate tax
DISPLAY lines, subtotal, discount, tax, and final total
```

7. Python Source Code

```
# Retail Billing with Multiple Discount Rules
products = [
    ("Notebook", 120, 10, 5),
    ("Backpack", 1200, 3, 12),
    ("Pen set", 250, 4, 5),
]

subtotal = 0
for name, price, quantity, tax_rate in products:
    subtotal += price * quantity

if subtotal >= 5000:
    discount = min(subtotal * 0.10, 1000)
else:
    discount = 0

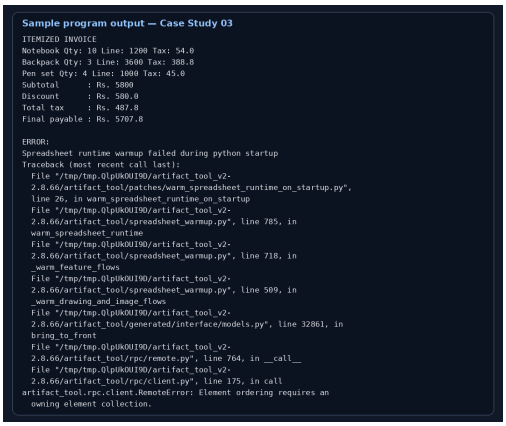
discount_ratio = discount / subtotal if subtotal > 0 else 0
total_tax = 0
print("ITEMIZED INVOICE")
for name, price, quantity, tax_rate in products:
    line_total = price * quantity
    discounted_line = line_total * (1 - discount_ratio)
    tax = discounted_line * tax_rate / 100
    total_tax += tax
    print(name, "Qty:", quantity, "Line:", round(line_total, 2),
          "Tax:", round(tax, 2))

final_total = subtotal - discount + total_tax
print("Subtotal      : Rs.", round(subtotal, 2))
print("Discount      : Rs.", round(discount, 2))
print("Total tax     : Rs.", round(total_tax, 2))
print("Final payable : Rs.", round(final_total, 2))
```

8. Test Cases

Test	Input / condition	Expected result
Above threshold	Sample basket subtotal ■5,800	10% discount = ■580, below the ■1,000 cap
Below threshold	Subtotal ■4,000	Discount ■0
Empty basket	No products	Subtotal and final total are ■0; no division error

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program creates an itemized invoice and demonstrates list processing, loops, conditional discounts, and tax calculations.

Case Study 04: Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a required attendance threshold.

2. Objectives

• Calculate current attendance accurately.

• Use a loop to find the minimum future classes required.

• Handle already-eligible and impossible inputs.

3. Assumptions

• Classes attended and total classes are non-negative integers; attended cannot exceed total.

• The required attendance threshold is 75%.

• Future scenario assumes every additional class is attended.

4. Algorithm

1. Set attended classes, total classes, and threshold.

2. Validate the input values.

3. Calculate current attendance percentage, or zero if total is zero.

4. If current percentage meets threshold, required classes are zero.

5. Otherwise increment future attended and total classes until the threshold is reached.

6. Display percentage and minimum classes needed.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
INPUT attended, total
IF attended < 0 OR total < 0 OR attended > total, DISPLAY invalid input
ELSE calculate current percentage
SET extra = 0
WHILE (attended + extra) / (total + extra) < threshold
  extra = extra + 1
DISPLAY current percentage and extra
```

7. Python Source Code

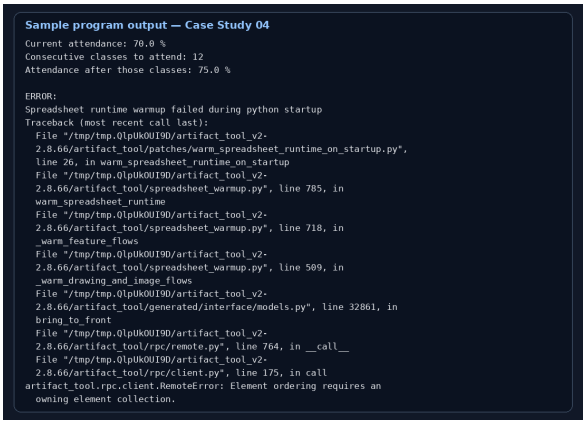
```
# Attendance Recovery Computation
attended = 42
total = 60
threshold = 75

if attended < 0 or total < 0 or attended > total:
    print("Invalid attendance data")
elif total == 0 and attended == 0:
    print("No classes recorded yet")
else:
    current = attended * 100 / total
    extra = 0
    while (attended + extra) * 100 / (total + extra) < threshold:
        extra += 1
    print("Current attendance:", round(current, 2), "%")
    if extra == 0:
        print("Already eligible.")
    else:
        print("Consecutive classes to attend:", extra)
    print("Attendance after those classes:",
          round((attended + extra) * 100 / (total + extra), 2), "%")
```

8. Test Cases

Test	Input / condition	Expected result
Already eligible	Attended 48 of 60	80%; required extra classes = 0
Needs recovery	Attended 42 of 60	70%; required extra classes = 12
Invalid	Attended 65; total 60	Invalid attendance data

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program uses a loop to find the smallest number of future attended classes that reaches the target percentage.

Case Study 05: Warehouse Inventory Reconciliation

1. Problem Statement

Reconcile opening stock, incoming stock, dispatched stock, and physically counted closing stock for multiple products. Identify discrepancies and calculate replenishment needs against target stock.

2. Objectives

• Calculate expected closing stock.

• Detect differences between expected and actual stock.

• Calculate replenishment needed to reach target stock.

3. Assumptions

• Each record is a tuple: product, opening, incoming, dispatched, actual closing, target stock.

• All quantities must be non-negative integers.

• Expected closing = opening + incoming – dispatched.

4. Algorithm

1. Read product records.

2. Validate quantities and prevent negative expected stock.

3. Calculate expected closing and compare it with actual closing.

4. Calculate discrepancy as actual minus expected.

5. Calculate replenishment as max(0, target minus actual).

6. Display reconciliation report.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
FOR each product record
  IF any quantity is negative or dispatched > opening + incoming
    DISPLAY invalid record
  ELSE
    expected = opening + incoming - dispatched
    discrepancy = actual - expected
    replenishment = maximum(0, target - actual)
    DISPLAY result
```

7. Python Source Code

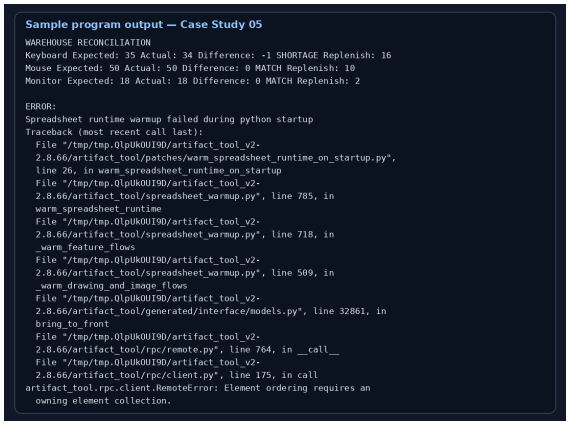
```
# Warehouse Inventory Reconciliation
records = [
    ("Keyboard", 50, 20, 35, 34, 50),
    ("Mouse", 80, 10, 40, 50, 60),
    ("Monitor", 25, 5, 12, 18, 20),
]

print("WAREHOUSE RECONCILIATION")
for name, opening, incoming, dispatched, actual, target in records:
    if min(opening, incoming, dispatched, actual, target) < 0:
        print(name, "- INVALID: negative quantity")
    elif dispatched > opening + incoming:
        print(name, "- INVALID: dispatch exceeds available stock")
    else:
        expected = opening + incoming - dispatched
        discrepancy = actual - expected
        replenishment = max(0, target - actual)
        if discrepancy == 0:
            status = "MATCH"
        elif discrepancy < 0:
            status = "SHORTAGE"
        else:
            status = "SURPLUS"
        print(name, "Expected:", expected, "Actual:", actual,
              "Difference:", discrepancy, status,
              "Replenish:", replenishment)
```

8. Test Cases

Test	Input / condition	Expected result
Matching stock	Opening 50, incoming 20, dispatched 35, actual 35	Expected 35; difference 0
Stock shortage	Expected 35, actual 34	Difference -1; shortage
Invalid dispatch	Opening 10, incoming 2, dispatched 15	Invalid: dispatch exceeds available stock

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program compares theoretical and physical stock, flags discrepancies, and estimates replenishment quantities.

Case Study 06: Password Policy Validation Engine

1. Problem Statement

Evaluate sample password strings against minimum length, uppercase, lowercase, digit, special-character, repeated-character, and prohibited-substring rules. Explain every failed condition.

2. Objectives

• Validate passwords using string operations and loops.

• Report all failed rules rather than stopping at the first failure.

• Classify a password as accepted or rejected.

3. Assumptions

• Minimum length is 8 characters.

• A password must contain uppercase, lowercase, a digit, and one special character from !@#$%&*.

• Three identical consecutive characters are prohibited; the substring 'password' is prohibited without case sensitivity.

4. Algorithm

1. Set sample passwords and policy rules.

2. For each password, create a list of failures.

3. Check length and character categories.

4. Check prohibited substring and three consecutive identical characters.

5. Print accepted or rejected and list failures.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
FOR each password
  SET failures to empty list
  CHECK length >= 8
  SCAN characters for uppercase, lowercase, digit, and special character
  CHECK prohibited substring
  CHECK every group of 3 consecutive characters
  IF failures empty, DISPLAY ACCEPTED
  ELSE DISPLAY REJECTED and failures
```

7. Python Source Code

```
# Password Policy Validation Engine
passwords = ["Kusuma@2026", "abc123", "Strong!!!9A", "password#9A"]

for password in passwords:
    failures = []
    if len(password) < 8:
        failures.append("must contain at least 8 characters")

    upper = lower = digit = special = False
    for ch in password:
        if ch.isupper():
            upper = True
        if ch.islower():
            lower = True
        if ch.isdigit():
            digit = True
        if ch in "!@#$%&*":
            special = True

    if not upper:
        failures.append("needs an uppercase letter")
    if not lower:
        failures.append("needs a lowercase letter")
    if not digit:
        failures.append("needs a digit")
    if not special:
        failures.append("needs a special character from !@#$%&*")
    if "password" in password.lower():
        failures.append("contains prohibited word 'password'")

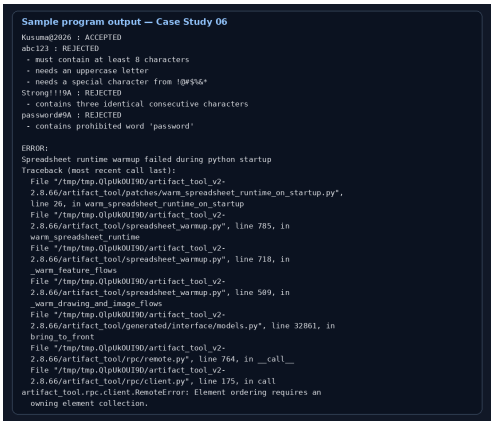
    repeated = False
    for i in range(len(password) - 2):
        if password[i] == password[i + 1] == password[i + 2]:
            repeated = True
    if repeated:
        failures.append("contains three identical consecutive characters")
```

```
if len(failures) == 0:
    print(password, ": ACCEPTED")
else:
    print(password, ": REJECTED")
    for failure in failures:
        print(" -", failure)
```

8. Test Cases

Test	Input / condition	Expected result
Strong password	Kusuma@2026	Accepted
Too short	abc123	Rejected for length and missing character categories
Repeated characters	Strong!!!9A	Rejected for three consecutive identical characters
Prohibited substring	password#9A	Rejected because the prohibited substring 'password' is present

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program demonstrates string methods, character scanning, and multiple validation rules. It is an educational example and is not a complete production security checker.

Case Study 07: Student Marks Matrix Analysis

1. Problem Statement

Analyse a nested list of marks for multiple students across multiple subjects. Calculate student totals and averages, subject averages, identify the top performer, and flag subjects with low average marks.

2. Objectives

• Use nested lists to store a marks matrix.

• Calculate row-wise and column-wise statistics.

• Identify top student and weak subjects.

3. Assumptions

• Marks are integers from 0 to 100.

• Every student has marks for the same subjects.

• A subject average below 50 is flagged as low.

4. Algorithm

1. Define student names, subject names, and marks matrix.

2. Validate each mark.

3. Calculate each student's total and average.

4. Calculate each subject's average using column positions.

5. Find top performer by total marks and flag low-average subjects.

6. Print summary.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
FOR each row in marks matrix, calculate total and average
FOR each subject column, add marks across all students and calculate average
FIND student with maximum total
FOR each subject, IF average < 50, flag as low
DISPLAY all statistics
```

7. Python Source Code

```
# Student Marks Matrix Analysis
students = ["Asha", "Bharat", "Charan", "Divya"]
subjects = ["Python", "Maths", "Computing"]
marks = [
    [88, 76, 91],
    [65, 72, 68],
    [42, 48, 55],
    [95, 89, 93],
]

valid = True
for row in marks:
    for mark in row:
        if mark < 0 or mark > 100:
            valid = False

if not valid:
    print("Invalid marks: values must be from 0 to 100")
else:
    totals = []
    print("STUDENT RESULTS")
    for i in range(len(students)):
        total = sum(marks[i])
        average = total / len(subjects)
        totals.append(total)
        print(students[i], "Total:", total, "Average:", round(average, 2))

    print("\nSUBJECT AVERAGES")
    for j in range(len(subjects)):
        total = 0
        for i in range(len(students)):
            total += marks[i][j]
        average = total / len(students)
        print(subjects[j], "Average:", round(average, 2),
              "LOW AVERAGE" if average < 50 else "")

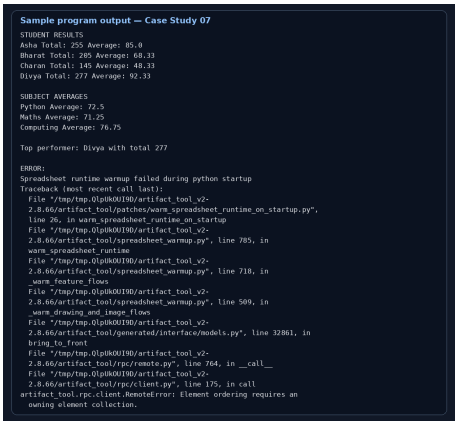
    top_index = totals.index(max(totals))
```

```
print("\nTop performer:", students[top_index],
      "with total", totals[top_index])
```

8. Test Cases

Test	Input / condition	Expected result
Normal matrix	All marks within 0–100	Totals, averages, subject averages, and top performer displayed
Low subject average	One subject average below 50	Subject flagged LOW AVERAGE
Invalid mark	Any mark = 105	Invalid marks message

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program demonstrates nested-list traversal and row-wise/column-wise calculations for academic performance analysis.

Case Study 08: Sports Tournament Ranking System

1. Problem Statement

Store match outcomes for several teams, calculate points and goal difference, and rank teams using points, goal difference, and goals scored as tie-breakers.

2. Objectives

• Represent match data with tuples.

• Calculate points from match results.

• Rank teams using clear tie-breaking rules.

3. Assumptions

• A win earns 3 points, a draw earns 1 point, and a loss earns 0 points.

• Match tuples contain home team, away team, home score, and away score.

• All scores are non-negative integers.

4. Algorithm

1. Define teams and match tuples.

2. Initialize points, goal difference, and goals scored for every team.

3. For each match, update goals scored and goal difference.

4. Award points based on result.

5. Sort teams by points, goal difference, and goals scored, all descending.

6. Display ranking.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
INITIALIZE statistics for each team
FOR each match tuple
  UPDATE goals scored and goal difference for both teams
  IF home score > away score, award 3 points to home
  ELSE IF scores equal, award 1 point to each
  ELSE award 3 points to away
SORT by points descending, goal difference descending, goals scored descending
DISPLAY ranking
```

7. Python Source Code

```
# Sports Tournament Ranking System
teams = ["Falcons", "Tigers", "Eagles", "Sharks"]
matches = [
    ("Falcons", "Tigers", 2, 1),
    ("Eagles", "Sharks", 1, 1),
    ("Falcons", "Eagles", 0, 1),
    ("Tigers", "Sharks", 3, 0),
    ("Falcons", "Sharks", 2, 2),
    ("Tigers", "Eagles", 1, 2),
]

# Parallel lists: points, goal difference, and goals scored
points = [0] * len(teams)
goal_difference = [0] * len(teams)
goals_scored = [0] * len(teams)

for home, away, home_score, away_score in matches:
    if home_score < 0 or away_score < 0:
        print("Invalid score in match:", home, away)
        continue
    h = teams.index(home)
    a = teams.index(away)
    goals_scored[h] += home_score
    goals_scored[a] += away_score
    goal_difference[h] += home_score - away_score
    goal_difference[a] += away_score - home_score

    if home_score > away_score:
        points[h] += 3
    elif home_score == away_score:
        points[h] += 1
        points[a] += 1
    else:
        points[a] += 3
```

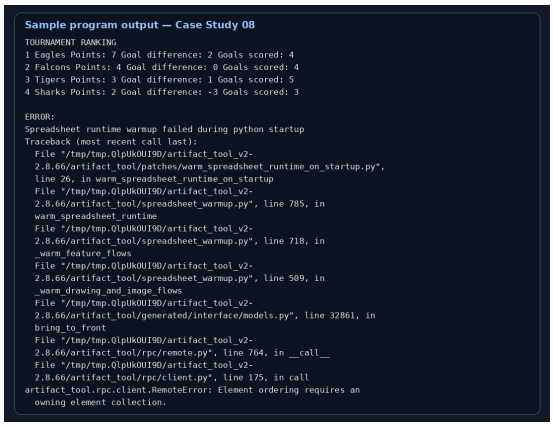
```
# Rank team indices by points, goal difference, then goals scored.
ranking = list(range(len(teams)))
for i in range(len(ranking)):
    for j in range(i + 1, len(ranking)):
        left = ranking[i]
        right = ranking[j]
        if (points[right] > points[left] or
            (points[right] == points[left] and goal_difference[right] > goal_difference[left]) or
            (points[right] == points[left] and goal_difference[right] == goal_difference[left]
             and goals_scored[right] > goals_scored[left])):
            ranking[i], ranking[j] = ranking[j], ranking[i]

print("TOURNAMENT RANKING")
for position, index in enumerate(ranking, start=1):
    print(position, teams[index], "Points:", points[index],
          "Goal difference:", goal_difference[index],
          "Goals scored:", goals_scored[index])
```

8. Test Cases

Test	Input / condition	Expected result
Win	Home score 2, away score 1	Home team gets 3 points
Draw	Score 1–1	Both teams get 1 point
Tie-break	Equal points	Goal difference, then goals scored determine order

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program uses tuples, parallel lists, nested loops, and conditional statements to create a deterministic tournament table without relying on dictionaries.

Case Study 09: Smart Canteen Order and Stock Management

1. Problem Statement

Process a sequence of canteen orders, calculate bills, update inventory, reject orders that exceed available stock, and summarize sales and remaining stock.

2. Objectives

• Maintain product stock in lists.

• Validate order quantities before accepting an order.

• Calculate sales and report remaining stock.

3. Assumptions

• Each order is a tuple of product name and quantity.

• Prices and initial stock are supplied in parallel lists.

• Only positive quantities and known product names are accepted.

4. Algorithm

1. Define menu items, prices, stock, and sample orders.

2. For each order, locate the product.

3. Reject unknown products, non-positive quantities, or quantities exceeding stock.

4. Otherwise calculate bill, reduce stock, and add to total sales.

5. Display each order result and final stock.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
SET menu, prices, stock, and orders
SET total_sales = 0
FOR each order
  FIND product index
  IF not found, reject
  ELSE IF quantity <= 0, reject
  ELSE IF quantity > stock, reject
  ELSE calculate bill, reduce stock, add bill to sales
DISPLAY total sales and remaining stock
```

7. Python Source Code

```
# Smart Canteen Order and Stock Management
menu = ["Idli", "Dosa", "Sandwich", "Juice"]
prices = [30, 50, 70, 40]
stock = [40, 25, 15, 20]
orders = [
    ("Idli", 4),
    ("Dosa", 2),
    ("Sandwich", 20),  # rejected: insufficient stock
    ("Juice", 3),
    ("Unknown", 1),    # rejected: not on menu
]

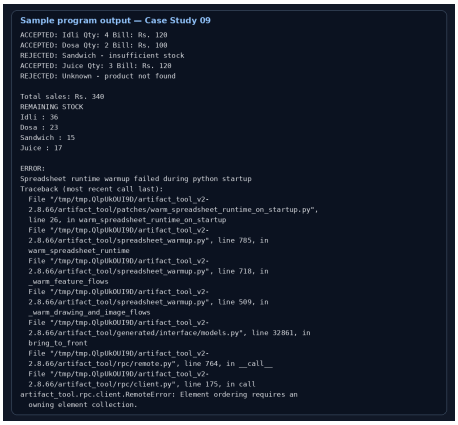
total_sales = 0
for product, quantity in orders:
    if product not in menu:
        print("REJECTED:", product, "- product not found")
    elif quantity <= 0:
        print("REJECTED:", product, "- quantity must be positive")
    else:
        index = menu.index(product)
        if quantity > stock[index]:
            print("REJECTED:", product, "- insufficient stock")
        else:
            bill = prices[index] * quantity
            stock[index] -= quantity
            total_sales += bill
            print("ACCEPTED:", product, "Qty:", quantity,
                  "Bill: Rs.", bill)

print("\nTotal sales: Rs.", total_sales)
print("REMAINING STOCK")
for i in range(len(menu)):
    print(menu[i], ":", stock[i])
```

8. Test Cases

Test	Input / condition	Expected result
Valid order	Idli quantity 4; stock 40	Accepted; bill ■120; stock becomes 36
Insufficient stock	Sandwich quantity 20; stock 15	Rejected; stock unchanged
Unknown item	Product 'Unknown'	Rejected: product not found
Invalid quantity	Quantity 0	Rejected: quantity must be positive

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program simulates order processing, stock control, and daily sales using lists, tuples, loops, and conditional statements.

Case Study 10: Integrated Student Record Consistency Checker

1. Problem Statement

Validate student identifiers, names, marks, attendance, and course records. Detect duplicate roll numbers, missing fields, invalid marks or attendance, and inconsistent records.

2. Objectives

• Validate records using strings, lists, and tuples.

• Identify duplicate identifiers and invalid values.

• Generate a reason-wise consistency report.

3. Assumptions

• Each record is a tuple: roll number, name, marks, attendance, course code.

• Roll numbers must be unique and non-empty.

• Marks must be between 0 and 100, attendance between 0 and 100, and name/course code must not be empty.

4. Algorithm

1. Define sample student tuples.

2. Scan records and count each roll number.

3. For each record, create an error list.

4. Check duplicate roll number, missing values, marks, attendance, and course code.

5. Display VALID or INVALID with reasons.

5. Flowchart

START → LOAD INPUTS → VALIDATE → PROCESS WITH CONDITIONS/LOOPS → DISPLAY RESULTS → END

6. Pseudocode

```
COUNT occurrences of each roll number
FOR each student tuple
  SET errors to empty list
  CHECK missing fields
  CHECK roll number count > 1
  CHECK marks range 0–100
  CHECK attendance range 0–100
  CHECK course code not empty
  DISPLAY VALID if no errors; otherwise display errors
```

7. Python Source Code

```
# Integrated Student Record Consistency Checker
records = [
    ("26000293", "Kusuma Priya", 92, 96, "CSE101"),
    ("26000294", "Yamuna", 78, 82, "CSE101"),
    ("26000293", "Duplicate Roll", 65, 75, "CSE102"),
    ("26000296", "", 108, 110, ""),
]

roll_numbers = []
for record in records:
    roll_numbers.append(record[0])

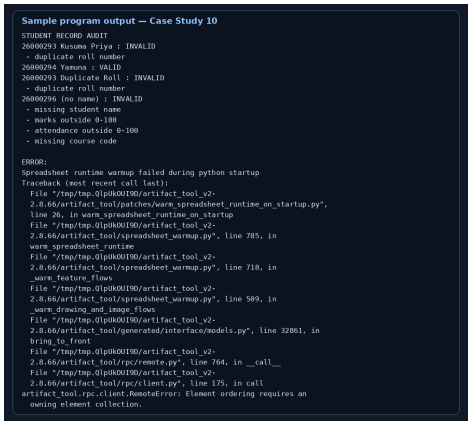
print("STUDENT RECORD AUDIT")
for roll, name, marks, attendance, course in records:
    errors = []
    if roll == "":
        errors.append("missing roll number")
    if roll_numbers.count(roll) > 1:
        errors.append("duplicate roll number")
    if name.strip() == "":
        errors.append("missing student name")
    if marks < 0 or marks > 100:
        errors.append("marks outside 0-100")
    if attendance < 0 or attendance > 100:
        errors.append("attendance outside 0-100")
    if course.strip() == "":
        errors.append("missing course code")

    if len(errors) == 0:
        print(roll, name, ": VALID")
    else:
        print(roll, name if name else "(no name)", ": INVALID")
        for error in errors:
            print(" -", error)
```

8. Test Cases

Test	Input / condition	Expected result
Valid record	Unique roll; name/course present; marks 92; attendance 96	VALID
Duplicate roll	Two records use roll 26000293	Both flagged duplicate
Invalid values	Marks 108; attendance 110; empty name/course	All corresponding errors reported

9. Output Screenshot / Sample Run



Captured output text is also supplied as sample_output.txt.

10. Conclusion

The program audits student records and explains consistency errors. The sample records are fictional and should not be treated as official student data.